FAI Final Project Report

Method Tried

期中考前基本探索過單純 feature based , monte carlo tree search , RL。但像是 RL 當時遇到的問題就是很難取得 trainning data、state space 不容易定義（太少會導致無法推理，太大又不易訓練），以及容易 overfit 在訓練時對手的策略。因此期中考後主要是在設法提昇 mcts 的能力，嘗試過的策略如下表所示：

策略描述	優點	缺點
引入 impossible_mask 記錄對手出牌歷史，剪除不可能的手牌分佈。	能有效避開顯而易見的爆牌列，防守力提升。	Timeout 極高，因頻繁的系統時間查詢與指派算法低效，常被判定 DQ。
測試深層樹搜尋與漸進式寬度擴展	在特定對手下能預測多步，ELO 有所上升。	深層搜尋反而降低了隨機盤面的泛化表現
稀釋時間檢測頻率；使用零分配陣列與無偏安全隨機 Rollout	運算效率大幅提升，超時率降至萬分之一	對對手第一步的策略建模仍有細化空間。

Core Idea

繳交的兩版都是基於將當前可能的牌發給對手作為其手牌，然後讓對手使用較為簡單易算的策略（如：greedy , featured based）來進行出牌，同時我們去觀察自己的手牌在這模擬對局中哪一張的效果最好來作為最後得出牌；差異主要體現為如何 sample 對手的手牌、 對手採取的策略、模擬的深度等。

BestPlayer1（無偏深度-2 ISMCTS）

- **對手策略**：採取混和方法，有 10% 的機率隨機出牌，其餘 90% 的情況下，評估手牌中哪些牌是「安全牌」（即放入牌桌後不會讓任何一列長度達到 6 的牌）。如果有多張安全牌，隨機選擇一張出牌。
- **手牌分配**：利用歷史局勢推導出一個 impossible_mask。如果某個對手在過去的某一輪中「可以選擇安全列，卻主動選擇了會爆掉的列出牌」，代表該對手當時手牌中絕對不包含任何能安全插入其他列的牌。
- **第一回合對手決策分析**：在搜尋樹的第一層轉折點，我們不對對手使用 Rollout，而是使用超幾何分佈估算（考慮間距內未知牌數量與對手手牌數量的機率尾端），快速推導出對手在當前盤面最可能選擇的安全卡牌。
- **探索深度**：強制在深度2時剪枝，以爭取更大的搜尋廣度。

BestPlayer2 （動態 MCTS）

- **對手策略**： 從第 2 輪開始，對手同樣採用類似 BestPlayer1 的策略；然而，如果搜尋進入了樹內節點，對手會根據該節點累積的勝率進行更有邏輯的樹狀抉擇，而非一味盲目 Rollout。
- **手牌分配**：每一步模擬都老實地執行對手手牌的隨機打亂與 mask 驗證，並有 10 次的重試機制。這在過程上產生的記憶體物件開銷較大。
- **第一回合對手決策分析**：和 BestPlayer1 一樣。
- **探索深度**：它沒有硬性規定層數，它會根據 in_tree 狀態無限向下生長。當遊戲進入中後期，剩餘未知牌極少時，樹會直接長到最底層（完全長滿）。

Implementation

為了解決 Python 在執行樹搜尋時的效能瓶頸，我們實作了多項優化：

1. 零記憶體分配陣列： 在蒙特卡羅模擬的 Rollout 階段，若頻繁宣告動態 list（如 safe_cards = []），會頻繁觸發 Python 的記憶體分配與回收，造成嚴重卡頓。我們將其重寫為：

```
safe_count = 0
safe_choices = [0] * len(hand)  # 預先分配固定長度陣列
# 遍歷卡牌，符合條件則填入 safe_choices[safe_count] 並遞增 safe_count
```

1. 第一步超幾何危險預估： 在第一層對手動作轉折點，使用超幾何分佈閉式解近似 MCTS 決策，公式為：

> $$P(Explode) = \Sigma_{i=capacity}^{min(K,\,O)} \frac{\binom{K}{i}\binom{U-K}{O-i}}{\binom{U}{O}}$$

其中 $capacity$ 為該列剩餘容量，K 為卡牌與該列結尾間的未知牌數，U 為總未知牌數，O 為對手手牌數。此算法以 $O(1)$ 時間完成了對手強度的建模。

Self-Assessment

做的順的地方

- 第一是有在網路上搜尋到 https://github.com/johannbrehmer/rl-6-nimmt/tree/master ，其實驗結果算是在一開始給予了我蠻好的設計方向。
- 助教給的建議都蠻實用的尤其是關於可以把對手強度設的更聰明而不是單純的 greedy，以及在鄰近比賽末應該可以從合理性就可以更好的去算對手手排可能性。

主要的困難

- 跑一次 tournament 的時間有點太久，即使已經開到工作站限制的 100 個 threads 仍然一次就大概要 1 小時左右，這使的調參會相當耗時。
- 很難去評估哪一個 agent 的效果比較好，因為我常常在本地測的結果跟拿去同學架的線上平台跑的結果兩者不一致，導致很難去判斷當前這樣寫是否是一個更好的方向。

Strategic Profiling

BestPlayer1

偏好的對手

- 多樣化混搭 / 隨機玩家
  原因：在大規模錦標賽或有隨機玩家亂入的牌局中，戰場極度混沌。此 agnent 不對未來做多步的理性假設（只看 2 步），後半段完全採用無偏隨機 Rollout。這反而有可能較貼近真實的混沌狀況。

不擅長的對手

- 其他寫的更深或更好的 MCTS
  原因：如果同桌全部都是極強的 MCTS 玩家，局勢高度有序。此時 Depth-2 缺陷就會暴露。當手牌剩下 4-5 張的收尾階段，高手們可能開始用深層樹完全求解、步步為營；而我們卻還在用隨機 Rollout 估算期望值，容易在多步連鎖反應中，被算力更深的高手精準設計，被迫接下滿載牛頭的牌列。

BestPlayer2

偏好的對手

- 高度理性 / 避險型 MCTS 玩家
  原因：這類對手的決策模式極其規律，都是以最小化自身爆牌機率為唯一目標。我們的深層樹搜尋在模擬時也假設對手是聰明的。這種雙方都是理性會達成博弈共識，讓我們能精準算計對手 3-5 步之後的動作。

不擅長的對手

- 完全隨機 / 極度混亂的玩家
  原因：隨機玩家會製造極大的環境噪音。當對手不按牌理出牌時，我們搜尋出來的深層預測鏈會因為對手的亂出牌而直接崩潰，導致其深層搜尋產生過度擬合，被混亂的局面誤傷。

Agent Comparison

兩個 agent 最大的 trade-offs 就是在搜索深度，BestPlayer1 其限制只有兩層這使的他的搜尋廣度較大，且不對長線做過於理想化的理性假設，這使其 robust 的程度相對較高；相應的其缺點是無法看穿多步以上的複雜卡位或連鎖陷阱，在大家都是高手時容易在多輪後的收尾階段被算力更深的對手精準做局，被迫吃下高分牌列，我對他的定位是提供一個穩定的地板。

BestPlayer2 搜尋樹會根據 UCB 的回饋動態向下生長，沒有深度限制。特別是在遊戲中後期，它能完全生長至葉子節點，進行確定性殘局求解；而其劣勢很明顯，過深的搜尋假設了對手「在資訊不透明下能做出完美決策」。如果模擬的卡牌分配稍有偏差，其表現容易有下降，我對此 agent 的定位是提供一個較有潛力的天花板。